

Q39. Minimum Spanning Tree vs Shortest Path

(Greedy Comparison)

Name: VANGIMALLA BALA SIVA PRASAD REDDY

Register number: 192424316

1. Aim

To compute the Minimum Spanning Tree (MST) and Shortest Path for a given weighted graph, compare their objectives and outputs, analyze their greedy selection strategies, evaluate their time and space complexity, and discuss their practical applications in graph optimization.

2. Problem Statement

Given a connected weighted undirected graph, the following problems are considered:

1. Find the Minimum Spanning Tree (MST) that connects all vertices with minimum possible total edge weight.
2. Find the Shortest Path between a specified source and destination vertex.
3. Compare the objectives, greedy strategies, outputs, correctness, computational complexity, and applications of both approaches.

The important point is that MST and Shortest Path solve different optimization problems, even though both can be solved using greedy algorithms.

3. Introduction

Graphs are widely used to represent networks such as roads, computer networks, communication systems, electrical grids, and transportation systems.

Two important graph optimization problems are:

- Minimum Spanning Tree: Finds a minimum-cost set of edges that connects every vertex.
- Shortest Path: Finds the minimum-cost route between a source and destination.

For MST, common greedy algorithms include Prim's Algorithm and Kruskal's Algorithm. For shortest path problems with non-negative edge weights, Dijkstra's Algorithm is a common greedy algorithm.

4. Given Graph

Consider the following weighted undirected graph:

Edge	Weight
A – B	4
A – C	2
B – C	1
B – D	5
C – D	8
C – E	10
D – E	2
D – F	6
E – F	3
B – E	7

We will calculate:

- MST for all vertices
- Shortest path from A to F

5. Minimum Spanning Tree

5.1 Definition

A Minimum Spanning Tree is a subset of edges of a connected weighted undirected graph that:

- Connects all vertices.
- Contains no cycles.
- Contains exactly $V - 1$ edges.
- Has the minimum possible total edge weight.

For the given graph: $V = 6$

Therefore: Number of MST edges = $V - 1 = 6 - 1 = 5$

6. Kruskal's Algorithm for MST

Kruskal's Algorithm is a greedy algorithm that sorts all edges by increasing weight and repeatedly selects the smallest edge that does not create a cycle.

Algorithm Steps

4. Sort all edges in increasing order of weight.
5. Select the smallest edge.
6. Check whether adding the edge creates a cycle.
7. If it does not create a cycle, add it to the MST.
8. If it creates a cycle, reject it.
9. Continue until $V - 1$ edges are selected.

6.1 Sorted Edges

Order	Edge	Weight
1	B – C	1
2	A – C	2
3	D – E	2
4	E – F	3
5	A – B	4
6	B – D	5
7	D – F	6
8	B – E	7
9	C – D	8
10	C – E	10

6.2 MST Selection

Step 1

Select B – C = 1. No cycle is formed.

Step 2

Select A – C = 2. No cycle is formed. Total = 1 + 2 = 3.

Step 3

Select $D - E = 2$. No cycle is formed. Total = $3 + 2 = 5$.

Step 4

Select $E - F = 3$. No cycle is formed. Total = $5 + 3 = 8$.

Step 5

Consider $A - B = 4$. A and B are already connected through $A \rightarrow C \rightarrow B$. Therefore, $A - B$ would create a cycle and is rejected.

Step 6

Consider $B - D = 5$. This connects the two separate components $A - B - C$ and $D - E - F$. Therefore, $B - D$ is selected.

Total MST cost: $1 + 2 + 2 + 3 + 5 = 13$

Five edges have now been selected, so the MST is complete.

7. MST Result

Edge	Weight
B – C	1
A – C	2
D – E	2
E – F	3
B – D	5

MST = $1 + 2 + 2 + 3 + 5 = 13$

Minimum Spanning Tree Cost = 13

8. Shortest Path

8.1 Definition

The Shortest Path problem finds the minimum-cost path between a specified source and destination vertex.

For this problem: Source = A; Destination = F

Since all edge weights are non-negative, Dijkstra's Algorithm can be used.

9. Dijkstra's Algorithm

Dijkstra's Algorithm is a greedy shortest-path algorithm. At each step, it selects the unvisited vertex having the smallest currently known distance from the source.

Algorithm Steps

10. Set the source distance to 0.
11. Set all other distances to infinity.
12. Select the unvisited vertex with the smallest distance.
13. Relax all of its adjacent edges.
14. Mark the selected vertex as visited.
15. Repeat until the destination is reached or all vertices are processed.
16. Use predecessor information to reconstruct the shortest path.

10. Dijkstra Calculation

Initial Distances

Vertex	Distance from A	Previous
A	0	—
B	∞	—
C	∞	—
D	∞	—
E	∞	—
F	∞	—

Step 1: Select A

A = 0. Relax $A \rightarrow B = 4$ and $A \rightarrow C = 2$.

Step 2: Select C

C = 2. C $\rightarrow$ B gives $2 + 1 = 3$, so B is updated from 4 to 3. C $\rightarrow$ D gives 10. C $\rightarrow$ E gives 12.

Step 3: Select B

$B = 3$. $B \rightarrow D$ gives $3 + 5 = 8$, so D is updated to 8. $B \rightarrow E$ gives $3 + 7 = 10$, so E is updated to 10.

Step 4: Select D

$D = 8$. $D \rightarrow E$ gives $8 + 2 = 10$, which does not improve E. $D \rightarrow F$ gives $8 + 6 = 14$.

Step 5: Select E

$E = 10$. $E \rightarrow F$ gives $10 + 3 = 13$. Since $13 < 14$, update F to 13.

11. Shortest Path Result

Trace the predecessor information: $F \leftarrow E \leftarrow B \leftarrow C \leftarrow A$

Therefore, the shortest path is: $A \rightarrow C \rightarrow B \rightarrow E \rightarrow F$

Total distance = $A-C + C-B + B-E + E-F = 2 + 1 + 7 + 3 = 13$

Shortest distance from A to F = 13

12. Comparison: MST vs Shortest Path

Feature	Minimum Spanning Tree	Shortest Path
Objective	Connect all vertices with minimum total cost	Find minimum-cost route between vertices
Main Algorithms	Prim's, Kruskal's	Dijkstra's, Bellman-Ford
Greedy Strategy	Select minimum-cost safe edge	Select vertex with minimum tentative distance
Output	Tree containing all vertices	Path between source and destination
Number of Edges	$V - 1$	Depends on selected route
Cycles	Not allowed	Normally no cycles in selected path
Source Vertex	Not required	Required
Destination Vertex	Not required	Required
Typical Application	Network design	Routing/navigation

Result for Given Graph	MST cost = 13	$A \rightarrow C \rightarrow B \rightarrow E \rightarrow F$, cost = 13
------------------------	---------------	---

13. Greedy Selection Strategy Comparison

13.1 MST Greedy Strategy

Kruskal's Algorithm follows: Smallest available edge $\rightarrow$ Check cycle $\rightarrow$ Select if safe $\rightarrow$ Continue

The algorithm focuses on selecting edges that safely contribute to connecting all components.

13.2 Shortest Path Greedy Strategy

Dijkstra's Algorithm follows: Select closest unvisited vertex $\rightarrow$ Relax adjacent edges $\rightarrow$ Update distances $\rightarrow$ Repeat

The algorithm focuses on finding the shortest distance from the source to each reachable vertex.

14. Correctness Analysis

14.1 Kruskal's Algorithm

Kruskal's Algorithm is correct because of the cut property. For a cut of a weighted graph, a minimum-weight edge crossing that cut can safely be included in an MST. Kruskal repeatedly selects the minimum-weight edge that does not form a cycle. Therefore, each selected edge is a safe choice. After $V - 1$ edges are selected, the resulting tree is a Minimum Spanning Tree.

14.2 Dijkstra's Algorithm

Dijkstra's Algorithm is correct when all edge weights are non-negative. When the algorithm selects the unvisited vertex with the smallest tentative distance, no later path through another unvisited vertex can produce a smaller distance. Therefore, the selected distance becomes final. Repeated relaxation then produces the shortest path.

15. Time Complexity

Kruskal's Algorithm

Sorting the edges requires $O(E \log E)$. Union-Find operations are nearly constant amortized time. Therefore, Time Complexity = $O(E \log E)$.

Dijkstra's Algorithm

Using an adjacency list and priority queue: Time Complexity = $O((V + E) \log V)$. For a connected graph, this is commonly expressed as $O(E \log V)$.

16. Space Complexity

Kruskal's Algorithm

Kruskal requires an edge list and a disjoint-set data structure. Therefore, Space Complexity = $O(V + E)$.

Dijkstra's Algorithm

Using an adjacency list requires Space Complexity = $O(V + E)$. Additional arrays are required for distance, visited status, and parent/predecessor.

17. Applications

Minimum Spanning Tree

- Computer network design
- Electrical grid design
- Telecommunications
- Road and railway network planning
- Water pipeline systems
- Connecting multiple buildings with minimum cable cost
- Clustering and network analysis

Example: If several college buildings need to be connected using network cables, MST can determine which cables should be installed so that every building is connected while minimizing the total cable cost.

Shortest Path

- GPS navigation
- Map routing
- Computer network routing
- Transportation systems

- Robotics
- Internet packet routing
- Game pathfinding
- Logistics and delivery planning

Example: If a delivery vehicle needs to travel from a warehouse to a customer, a shortest-path algorithm can determine the route with minimum travel distance or cost.

18. Important Insight

The most important distinction is: MST does not necessarily give the shortest path between two vertices.

An MST minimizes the total weight of the entire connecting tree. The path between two particular vertices in that tree may not be the shortest path available in the original graph.

Similarly, a collection of shortest paths does not necessarily form an MST. This happens because MST and Shortest Path optimize different objectives.

19. Results

Minimum Spanning Tree

- $B - C = 1$
- $A - C = 2$
- $D - E = 2$
- $E - F = 3$
- $B - D = 5$

MST Weight = 13

Shortest Path

Source: A

Destination: F

Path: $A \rightarrow C \rightarrow B \rightarrow E \rightarrow F$

Cost: $2 + 1 + 7 + 3 = 13$

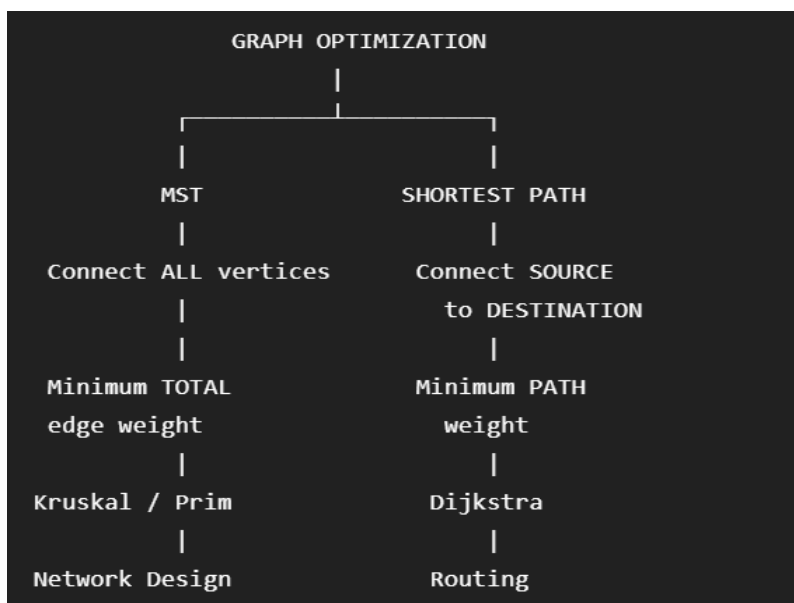
Shortest Path Cost = 13

In this particular graph, both values happen to be 13, but this is coincidental. It does not mean that MST and Shortest Path are equivalent.

20. Overall Comparison

Graph Optimization

- MST: Connect all vertices with minimum total edge weight → Kruskal/Prim → Network Design
- Shortest Path: Connect a source to a destination with minimum path weight → Dijkstra → Routing



21. Conclusion

Minimum Spanning Tree and Shortest Path are two important graph optimization problems that use greedy strategies but solve different objectives. Kruskal's Algorithm constructs an MST by selecting the smallest safe edges while avoiding cycles. Dijkstra's Algorithm finds the shortest path by repeatedly selecting the unvisited vertex with the smallest known distance and relaxing its neighboring edges.

For the given graph, the MST has a total weight of 13, while the shortest path from A to F is $A \rightarrow C \rightarrow B \rightarrow E \rightarrow F$, also having a cost of 13.

Although their costs are equal in this example, their purposes are fundamentally different. MST is suitable for designing a minimum-cost network connecting all nodes, whereas Shortest Path is suitable for finding an efficient route between specific nodes.

Therefore, the correct greedy algorithm must be selected according to the optimization objective. MST focuses on global network connectivity, while Shortest Path focuses on efficient movement between vertices.